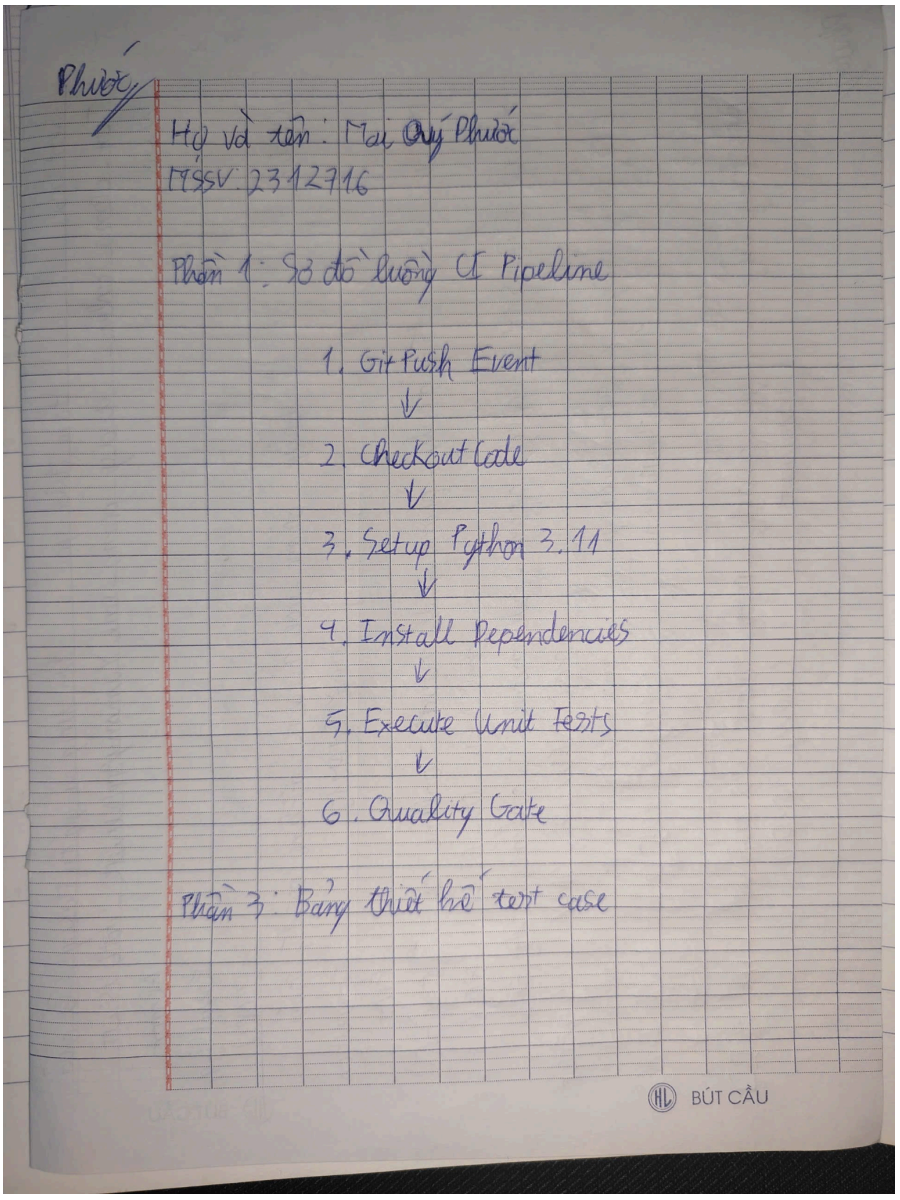
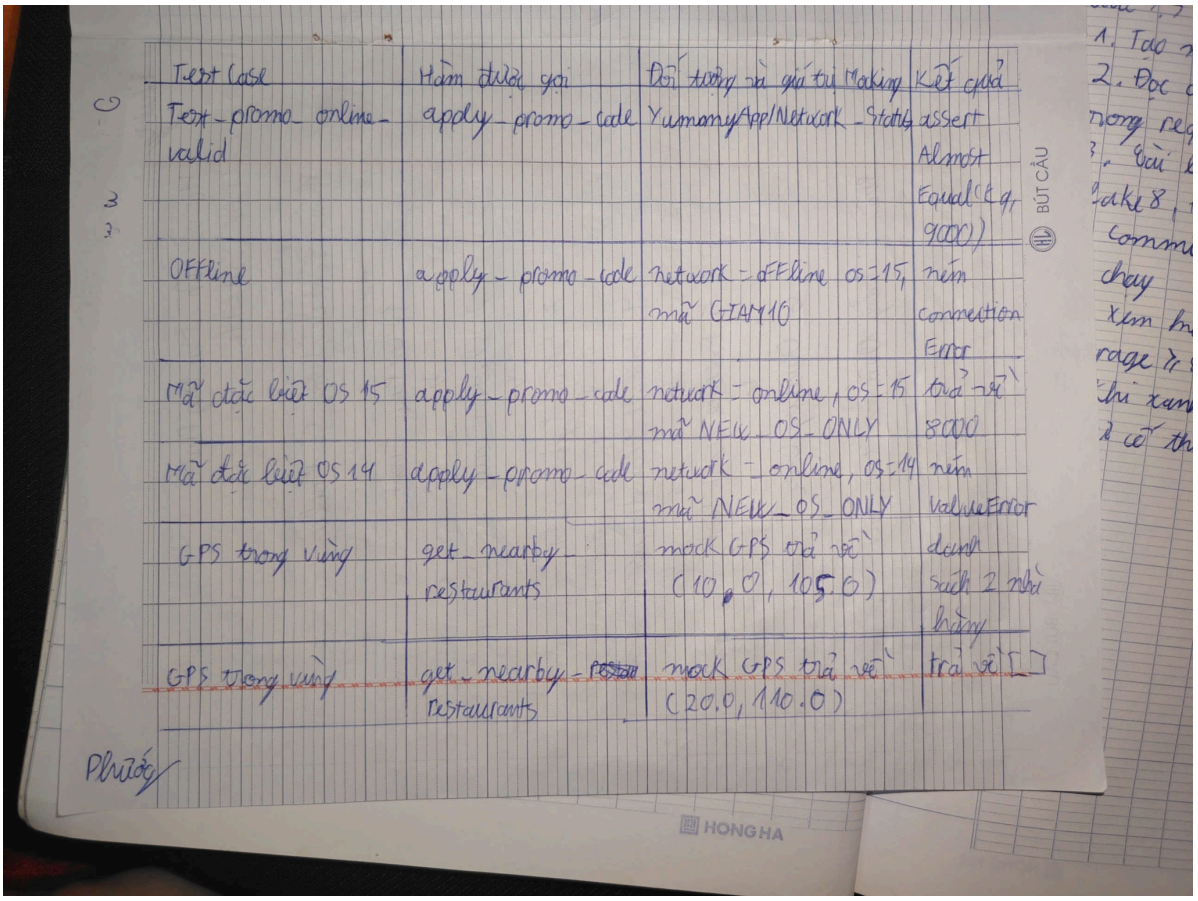
BÀI THỰC HÀNH 7

Kiểm thử Mocking ứng dụng di động và Tích hợp liên tục (CI/CD)

Phần 1. Sơ đồ luồng CI Pipeline

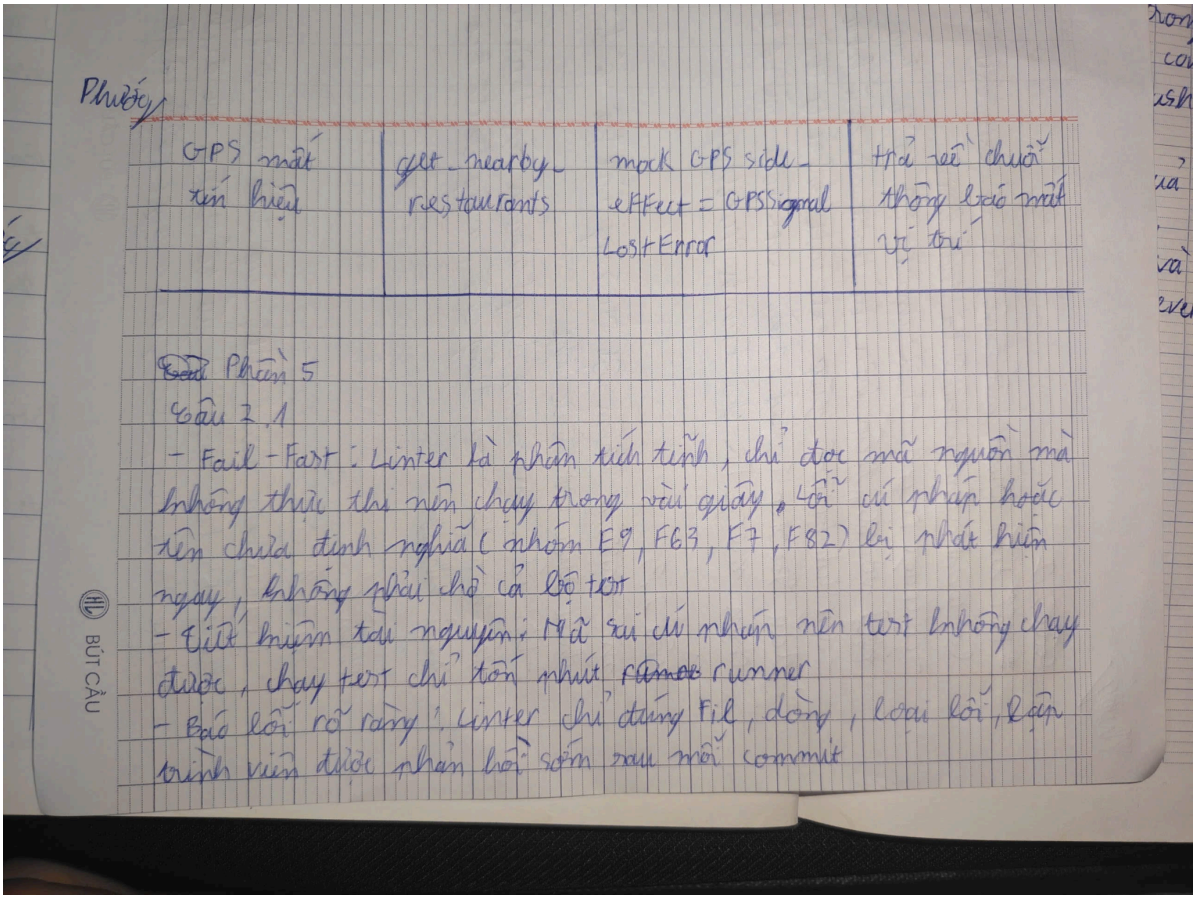


Phần 3. Bảng thiết kế Test Case



Test Case	Hàm được gọi	Đối tượng và giả trị Making	Kết quả
Test_promo_online_valid	apply_promo_code	YumanyApp/Network_Stath	assert Almost Equal(Eq, 9000)
OFFline	apply_promo_code	network = OFFline, os = 15, mã GIAM10	ném connection Error
Mã đặc biệt OS 15	apply_promo_code	network = online, OS = 15 mã NEW_OS_ONLY	trả về 8000
Mã đặc biệt OS 14	apply_promo_code	network = online, OS = 14 mã NEW_OS_ONLY	ném ValueError
GPS trong vùng	get_nearby_restaurants	mock GPS trả về (10.0, 105.0)	danh sách 2 nhà hàng
GPS trong vùng	get_nearby_restaurants	mock GPS trả về (20.0, 110.0)	trả về []

Phước



Phước

GPS mất tín hiệu	get_nearby_restaurants	mock GPS side_effect = GPSSignalLostError	trả về chuỗi thông báo mất vị trí

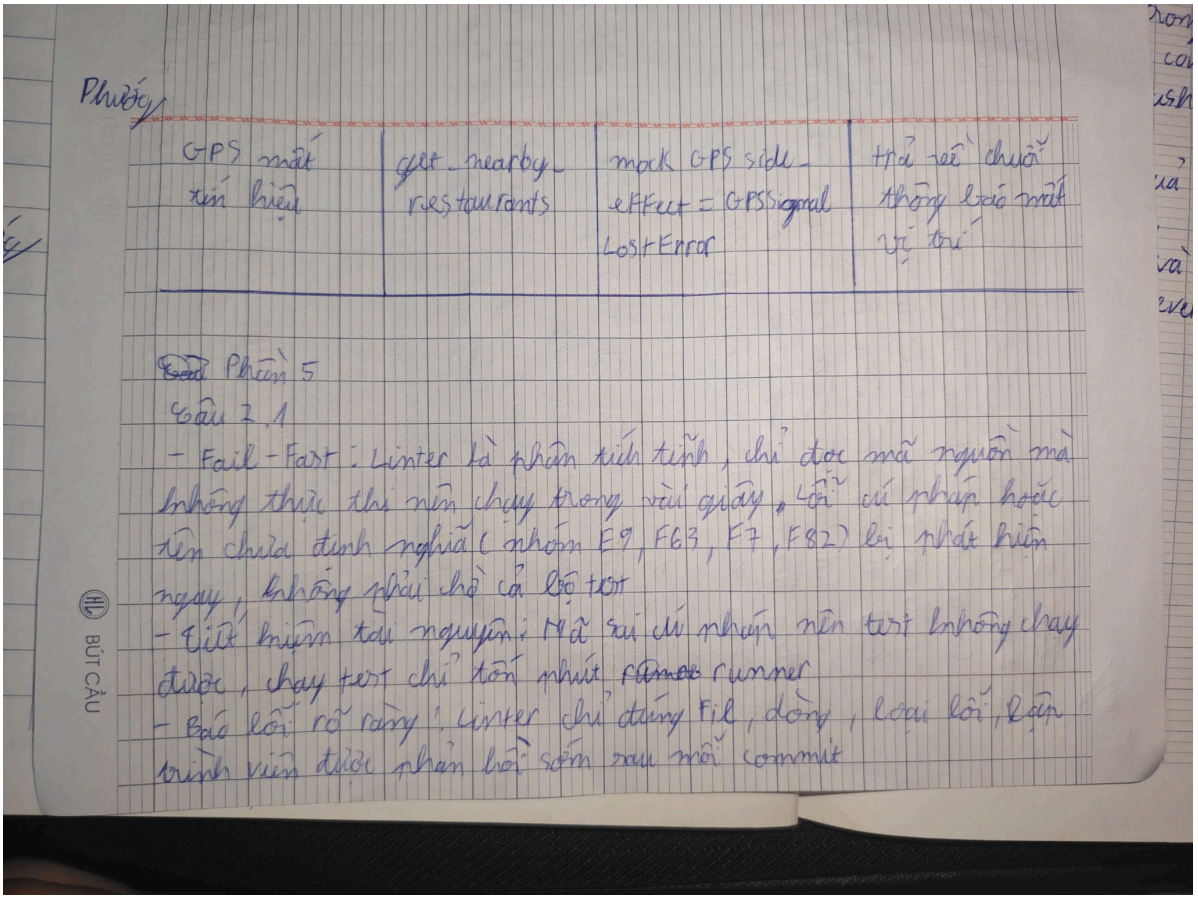
Phần 5

Câu 2.1

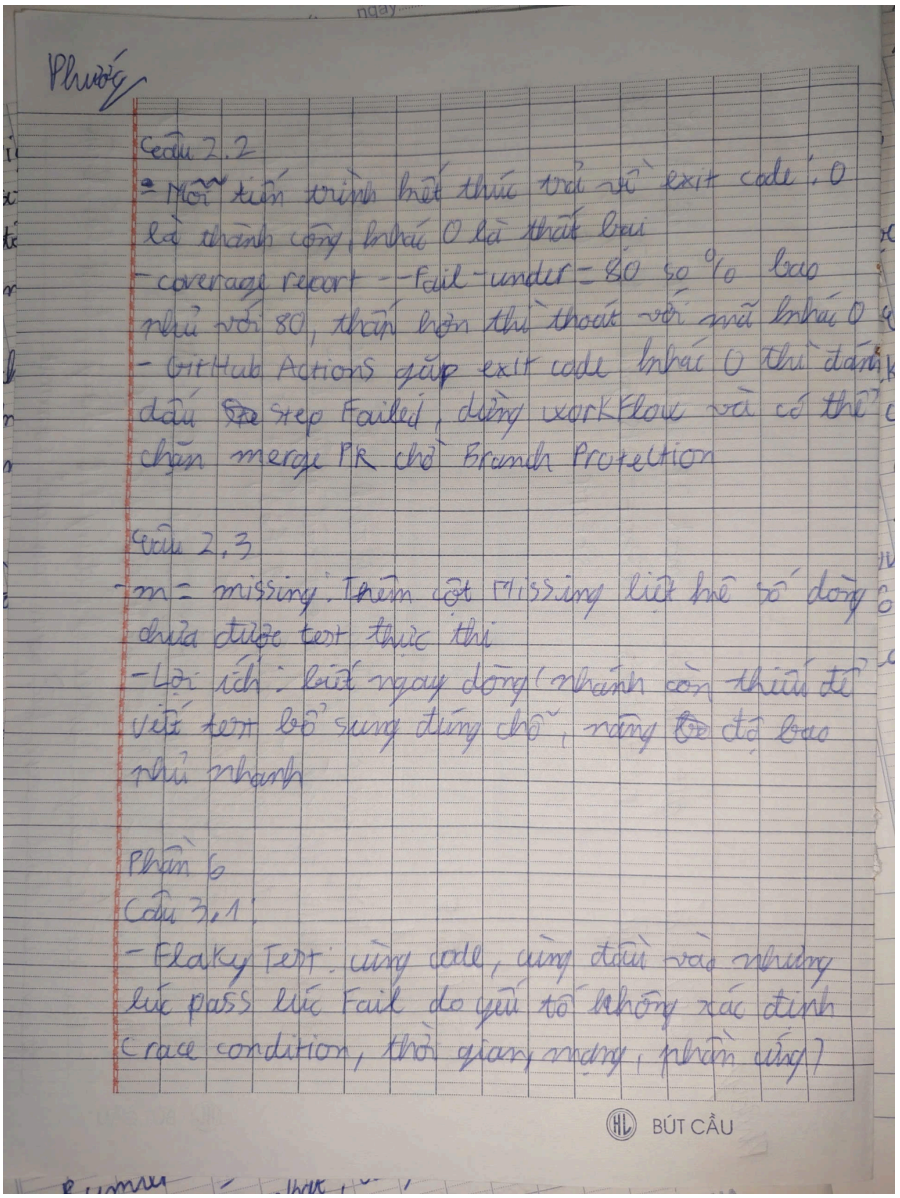
- Fail-Fast: Linter là nhân viên tỉnh, chỉ đọc mã nguồn mà không thực thi nên chạy trong vài giây, lỗi cú pháp hoặc tên chưa định nghĩa (nhóm E9, F63, F7, F82) bị phát hiện ngay, không phải chờ cả bộ test
- Tiết kiệm tài nguyên: Mã sai cú pháp nên test không chạy được, chạy test chỉ tốn phút runner
- Báo lỗi rõ ràng: Linter chỉ đúng File, dòng, loại lỗi, lập trình viên được phản hồi sớm sau mỗi commit

Phần 5.

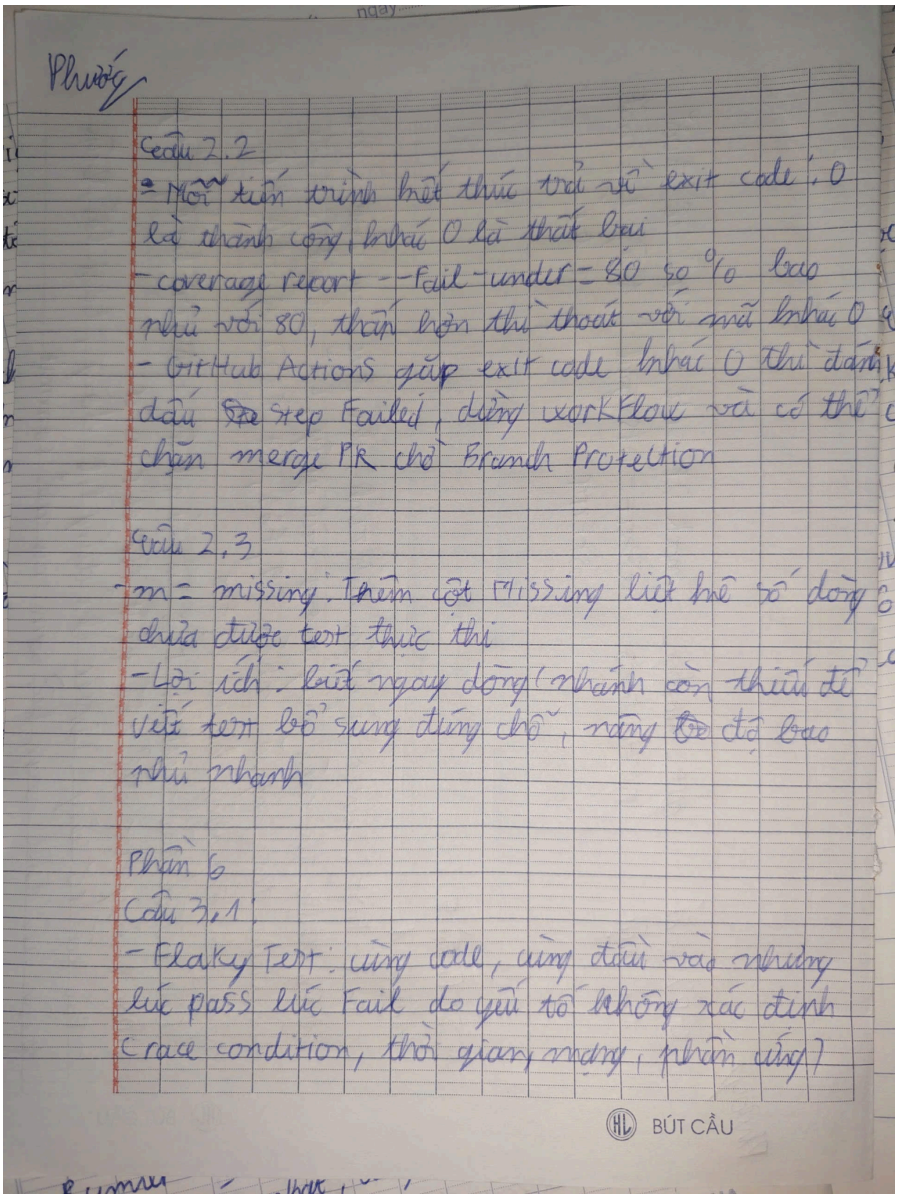
Câu 2.1. Tại sao Linter (flake8) phải đặt trước bước chạy test?



Câu 2.2. Nguyên lý hoạt động của --fail-under=80

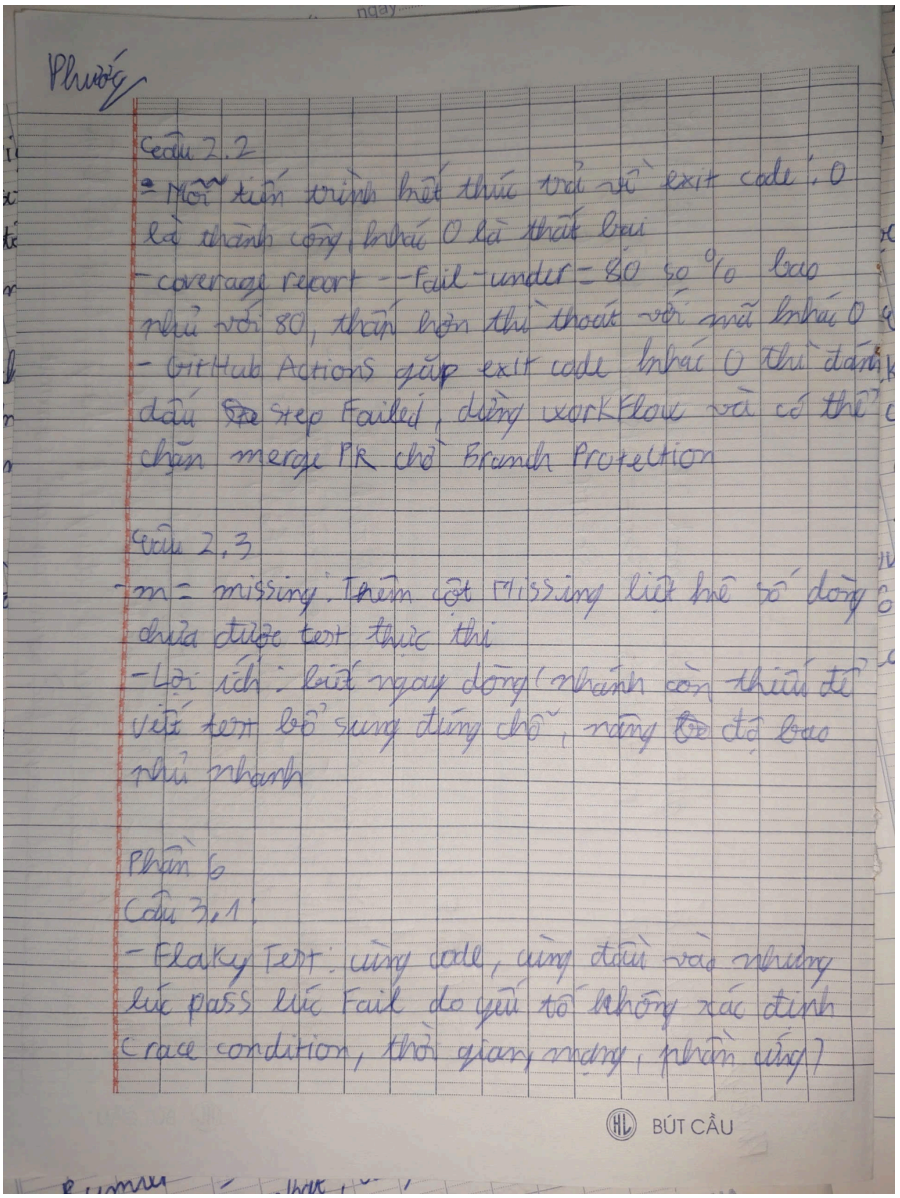


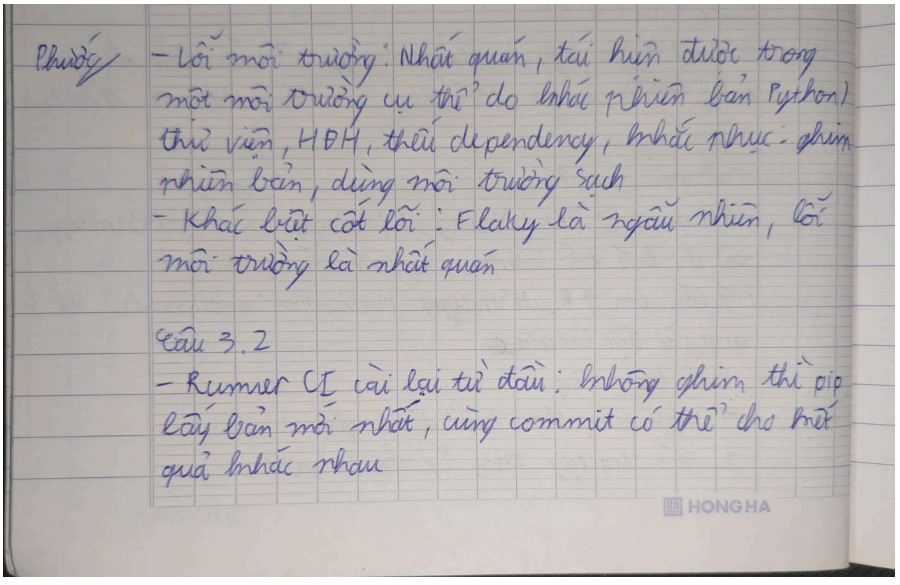
Câu 2.3. Tham số -m trong coverage report -m



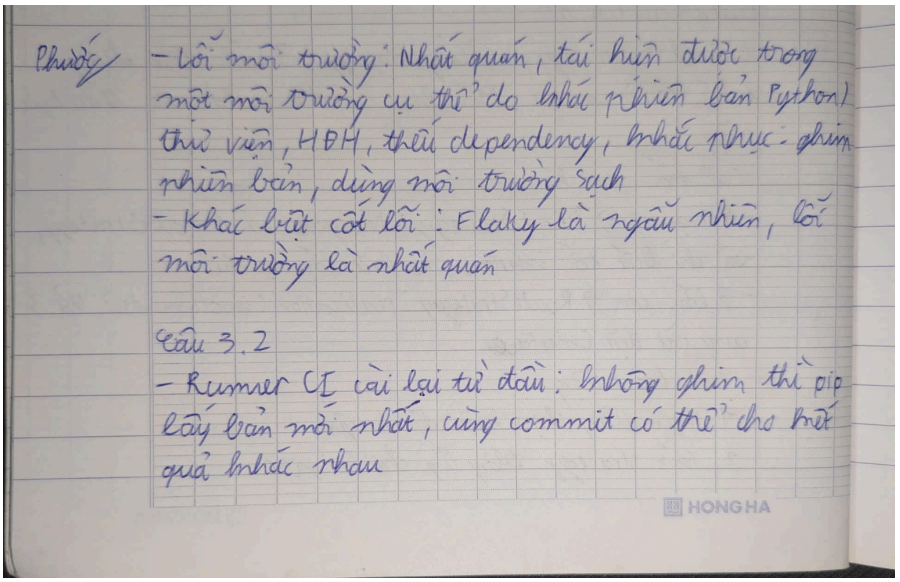
Phần 6.

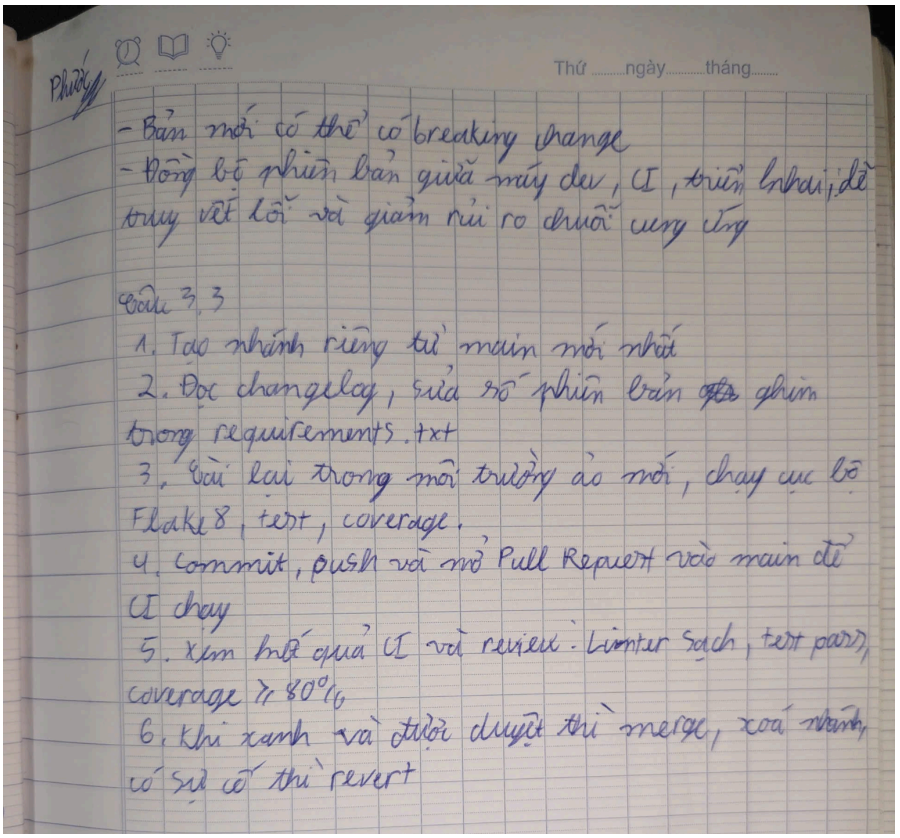
Câu 3.1. Phân biệt Flaky Test và lỗi do sai lệch môi trường phụ thuộc





Câu 3.2. Tại sao phải ghim phiên bản (Version Pinning) trong requirements.txt?





Câu 3.3. Quy trình 6 bước nâng cấp thư viện an toàn qua Pull Request

Phiếu

Thứ .....ngày.....tháng.....

- Bản mới có thể có breaking change
- Đồng bộ phiên bản giữa máy dev, CI, triển khai, để truy vết lỗi và giảm rủi ro chuỗi cung ứng

Câu 3.3

1. Tạo nhánh riêng từ main mới nhất
2. Đọc changelog, sửa số phiên bản ~~gfhm~~ ghim trong requirements.txt
3. Cài lại trong môi trường ảo mới, chạy cục bộ Flake8, test, coverage.
4. Commit, push và mở Pull Request vào main để CI chạy
5. Xem kết quả CI và review: Linter sạch, test pass, coverage ≥ 80%
6. Khi xanh và đủ duyệt thì merge, xoá nhánh, có sự cố thì revert